rpi-videoplayer

A fully autonomous, zero-interaction video kiosk for Raspberry Pi 3 / 4 / 5

Raspberry Pi · Python 3 · Flask · Open Source

> **What it does in one sentence:** The Raspberry Pi boots directly into fullscreen video playback, loops your videos endlessly, and simultaneously hosts its own WiFi hotspot so you can manage the playlist from any phone or laptop — no keyboard, mouse, or internet required.

System Overview

At its core, rpi-videoplayer is three coordinated systemd services that start automatically on boot:

The web service and display service both start after the access point is up, ensuring the network is ready before the UI is served.

Service 1 — WiFi Access Point

`videoplayer-ap.service`

The Pi creates its own WiFi network using `hostapd` (the access point daemon) and `dnsmasq` (DHCP + DNS server). No external router or internet connection is needed.

Setting	Value
WiFi SSID	`VideoPlayer`
Password	`videoplayer123`
Pi's IP address	`192.168.4.1`
Client IP range	`192.168.4.10 – 192.168.4.50`
Web UI URL	`http://192.168.4.1`

When a device connects to `VideoPlayer`, it receives an IP from the DHCP pool, and the DNS server resolves `videoplayer.local` back to `192.168.4.1` for convenience.

Service 2 — Flask Web Interface

`videoplayer-web.service`

A lightweight Flask application runs on **port 80** (no root password needed — the service runs as root for port binding). It provides a browser-based UI to manage the video playlist.

API Endpoints

Route	Method	Action
`GET /`	GET	Show playlist and player status
`/upload`	POST	Upload one or more video files (up to 4 GB)
`/delete/<filename>`	POST	Remove a video from the playlist
`/reorder`	POST (JSON)	Save a new drag-and-drop order
`/restart`	POST	Force immediate playlist reload
`/api/status`	GET	JSON status: playing/idle + video count

Playlist Persistence

The web server stores the user-defined playback order in `/opt/videoplayer/playlist_order.json`. When new videos are uploaded that aren't yet in the order file, they are appended alphabetically at the end — so the order is never lost.

Triggering Playback Reload

Every write operation (upload, delete, reorder) touches the file `/opt/videoplayer/.reload`. The player process watches for this file and immediately restarts its playlist loop when it appears.

Service 3 — Video Display

`videoplayer-display.service`

This service launches the display stack directly via DRM/KMS — no X11 or desktop environment required:

The service runs as the `videoplayer` system user. mpv outputs directly to the GPU framebuffer, bypassing any window manager.

player.py — The Playlist Engine

`player.py` is the heart of the system. It continuously loops through all video files and plays them one-by-one using **mpv** with hardware decoding enabled.

```
MPV_ARGS = [
    "mpv", "--fullscreen", "--no-osc",
    "--no-input-default-bindings", # disable keyboard shortcuts
    "--hwdec=auto",                # GPU hardware decoding
    "--gpu-context=drm",           # direct rendering (no X11)
    "--video-sync=display-resample", # smooth sync
]
```

The player never stops — when the last video finishes, it starts again from the top.

Live Filesystem Watching

`player.py` uses the **watchdog** library to watch two locations simultaneously:

VideoFolderHandler

Watches `/opt/videoplayer/videos/`

Any file created, deleted, or moved with a supported extension immediately sets the *reload event*, interrupting the current video and restarting the playlist.

ReloadFlagHandler

Watches `/opt/videoplayer/` for `.reload`

When the web server touches this file, the reload event fires — the player finishes killing mpv and rebuilds the playlist from scratch.

Placeholder Behavior

If no videos exist in the folder, the player shows a static placeholder image (`placeholder.jpg`) using mpv's image display mode, and keeps checking for new files via the filesystem watcher.

File Structure

```
/opt/videoplayer/ ├── player.py # Playlist engine (Python) ├── ap-start.sh # Brings up the WiFi AP ├── ap-stop.sh # Tears down the WiFi AP ├── playlist_order.json # Persisted drag-and-drop order ├── .reload # Touch this file to force reload ├── logs/ │ ├── player.log # Video playback events │ ├── web.log # Web server requests/errors │ └── display.log # Service output ├── videos/ # ← Drop video files here └── web/ ├── app.py # Flask web server (port 80) ├── static/ │ └── placeholder.jpg # Shown when playlist is empty └── templates/ └── index.html # Web UI
```

Supported Video Formats

Extension	Notes
`.mp4`	**Recommended** — best hardware decode performance on RPi GPU
`.mkv`	Matroska container, wide codec support
`.avi`	Legacy format, fully supported
`.mov`	Apple QuickTime container
`.webm`	VP8/VP9 — software decode may be needed
`.ts`	MPEG Transport Stream
`.m4v`	iTunes Video format

> **Best Practice:** Use **H.264 MP4** files for the smoothest playback across all models. The Raspberry Pi GPU can decode H.264 entirely in hardware, leaving the CPU free and preventing frame drops.
>
> **RPi 3 note:** Stick to H.264 content at 1080p or lower. RPi 3's VideoCore IV has limited hardware decoding support in KMS/DRM mode and will fall back to software decoding for H.265.

Installation Flow

The single `install.sh` script performs these steps when run as root:

1. **Install packages** — `mpv`, `python3`, `flask`, `hostapd`, `dnsmasq`, `openbox`, `unclutter`, `watchdog`
2. **Create service user** — `videoplayer` (in groups: video, audio, input, tty, render)
3. **Copy files** to `/opt/videoplayer/`
4. **Configure WiFi AP** — write `hostapd.conf`, `dnsmasq.conf`, and configure network stack
5. **Configure Openbox** — write `autostart` and `.xinitrc` for the service user
6. **Create systemd services** — `videoplayer-ap`, `videoplayer-web`, `videoplayer-display`
7. **Enable services** — all three start automatically on every boot
8. **Suppress boot splash** — disables RPi rainbow screen and verbose boot messages

Quick Usage Guide

1. Connect to WiFi

On your phone or laptop, connect to:

SSID: `VideoPlayer`
Password: `videoplayer123`

2. Open Web UI

In your browser, go to:

`http://192.168.4.1`

The playlist manager loads immediately.

3. Upload Videos

Tap the upload area or drag files in. Multiple files can be uploaded at once. Playback updates within seconds.

4. Manage Playlist

Drag rows to reorder. Click the trash icon to delete. The order persists across reboots.

Troubleshooting

Symptom	Command to diagnose
No WiFi AP visible	`sudo systemctl status videoplayer-ap`
Web UI not loading	`tail -f /opt/videoplayer/logs/web.log`
Black screen / no video	`tail -f /opt/videoplayer/logs/display.log`
Video stuttering (RPi 3)	Switch to H.264 MP4 format, 1080p or lower
Video stuttering (RPi 4/5)	Switch to H.264 or H.265 MP4 format
RPi 5 WiFi issues	Check `dmesg` — interface may be `wlan1`
Force playlist reload	`touch /opt/videoplayer/.reload`

Restart all services:

```
sudo systemctl restart videoplayer-ap videoplayer-web videoplayer-display
```

Change WiFi SSID or password:
Edit `/etc/hostapd/hostapd.conf`, then run `sudo systemctl restart hostapd`.